

Análisis arquitectura — Producción Virgilio

Diagnóstico y recomendaciones sobre el archivo **index.html** (versión original, sin modificaciones). — 14/05/2026

Consulta del cliente

¿Hay ventaja de que esté en GitHub o que esté en un servidor que esté en el mismo galpón donde los operarios envían los mensajes desde su celular?

1) ¿Me podrías explicar por dónde viajan los mensajes que hacen en la aplicación y llegan al programa que está alojado en la página? Si la respuesta es por WiFi, ¿existe forma de mejorar esto, como por ejemplo llevar el server para que esté máximo 50 metros de los puestos de trabajo o aumentar con repetidores la señal de WiFi? Contame qué velocidad debería tener como mínimo para garantizar un buen funcionamiento.

2) Te voy a contar cómo estamos funcionando hoy en **Virgilio** (programa Producción Virgilio + programa Logística Virgilio que usa los datos del Sheet). Corregime si no es así: los operarios envían mensajes ingresando a la página Producción Virgilio alojada en GitHub Pages; desde la página, dichos mensajes son enviados a un Google Sheet (a través de WiFi) a un archivo de Google Sheet llamado "Producción Virgilio". Esta información es requerida por otra aplicación llamada Logística Virgilio (portable), realizada con código de Claude, instalada en una PC a 50 metros del galpón donde los operarios realizan los mensajes; este programa se comunica con el Google Sheet a través de Internet alámbrico y utiliza esos datos para hacer cálculos y dar informes sobre consultas que queramos realizar.

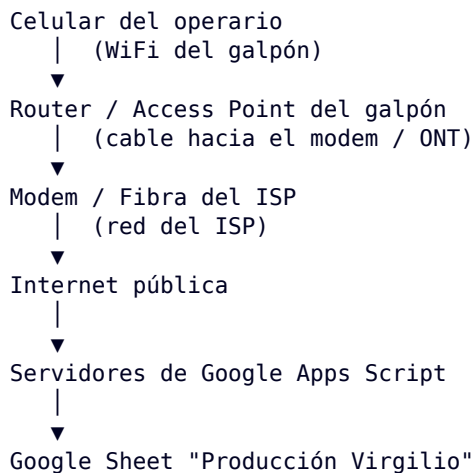
Ahora cómo lo hacemos en **Cervantes** (programa Registro Producción + programa Gestión Productiva que usa los datos en Supabase). Corregime si no es así: los operarios envían mensajes ingresando a la página Registro Producción alojada en GitHub Pages; desde la página los mensajes son enviados a Supabase (vía WiFi) a una base de datos llamada "Gestión Productiva", a la tabla "Registros Producción Cervantes". Esta información es requerida por otra aplicación llamada Gestión Productiva, creada con Claude, instalada en una PC a 5 metros de donde los operarios realizan los mensajes; se comunica con Supabase a través de Internet alámbrico, utiliza estos datos para hacer cálculos e informes.

3) Hay mensajes que no llegan. Como solución: que se guarden los mensajes enviados localmente, aparte de mandarlo, y que revise que no se pudo mandar y lo envíe nuevamente cuando ya tenga una señal de Internet estable.

1) Por dónde viajan los mensajes y si conviene mover el servidor

Esto es lo más importante y suele ser donde hay confusión: GitHub Pages **no recibe los mensajes**. GitHub Pages solo entrega el archivo **index.html** la primera vez que el celular abre la página. Después, los reportes viajan directo a Google, sin volver a pasar por GitHub.

Camino real de cada reporte



Entonces, ¿conviene moverlo al galpón?

Mover **GitHub Pages** al galpón **no soluciona casi nada**, porque GitHub solo sirve la página una vez. Lo que está lejos de verdad es **Google** (los servidores de Apps Script y Sheets viven en datacenters de Google, no en GitHub).

Para que mover el servidor sirva, tendrías que mover también **el backend** (lo que hoy es Apps Script + Sheets). Eso significa un servidor local en el galpón que reciba los mensajes y los guarde en una base local, y después sincronice a la nube. Es lo más resistente, pero también lo más complejo de mantener.

El cuello de botella real es el WiFi

Cada reporte mide ~300 bytes. La banda ancha **no es el problema**: 10 operarios mandando 10 reportes por minuto son menos de 50 KB/min. Lo que falla en galpones es:

- **Señal débil** detrás de estanterías metálicas, paredes gruesas y racks.
- **Pérdida de paquetes** (packet loss): el reporte se "corta" a mitad de camino.
- **Roaming malo**: el celular se queda pegado a un AP lejano en lugar de saltar al cercano.

Velocidad mínima razonable

- **Internet del galpón hacia afuera**: 10 Mbps de subida (upload) es más que suficiente. Lo importante es que sea **estable**, no rápido.
- **WiFi en el puesto del operario**: señal de **-65 dBm o mejor** (medible con apps como "WiFi Analyzer" en Android). Pérdida de paquetes <1%. Latencia hacia 8.8.8.8 (DNS de Google) <80 ms.

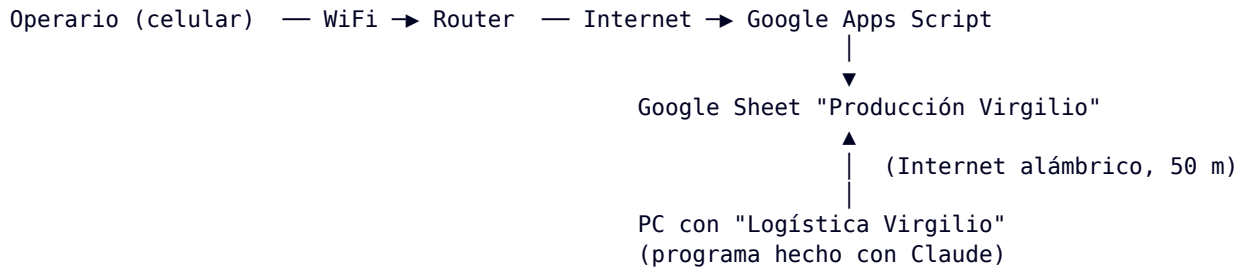
Mejoras concretas de WiFi, por orden de costo/beneficio

- 1 NO usar repetidores tradicionales.** Cortan la velocidad a la mitad y empeoran la latencia. Si ya tenés, sacalos.
- 2 Poner varios Access Points cableados** (no repetidores) con el **mismo SSID y contraseña**, configurados en canales distintos. Esto se llama "roaming". Para 50 m de galpón con estanterías metálicas, **2 o 3 APs alcanzan**.
- 3** Marcas que andan bien para este uso: **Ubiquiti UniFi, TP-Link Omada, MikroTik**. Los APs industriales arrancan en USD 80-150 c/u.
- 4 2.4 GHz penetra más** que 5 GHz a través de metal y mampostería. Si los operarios están lejos del AP, conviene activar 2.4 GHz aunque sea más lento — para 300 bytes da igual.
- 5** Una IP fija para cada AP y un solo SSID en todo el galpón, así el celular salta solo entre APs cuando el operario camina.

2) Cómo funcionan hoy Virgilio y Cervantes

Tu entendimiento **es correcto en lo esencial**. Algunas precisiones técnicas:

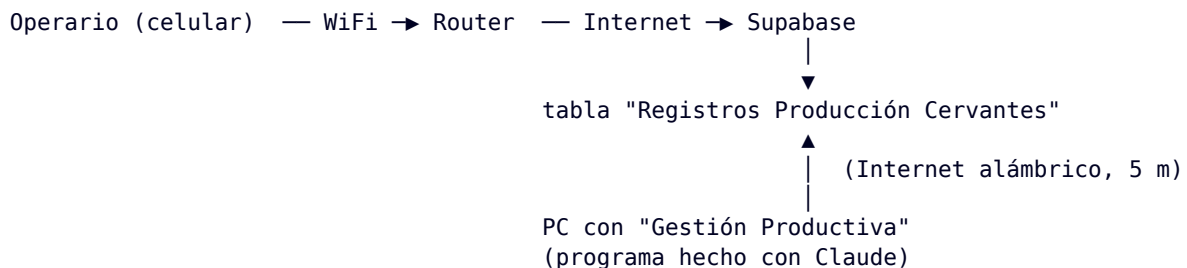
Virgilio (estado actual)



Aclaraciones:

- **GitHub Pages sirve el HTML una sola vez** al abrir la app, no en cada reporte.
- Entre la página y el Sheet **hay un paso intermedio**: Google Apps Script. Es un script de Google que recibe el POST y escribe la fila en el Sheet. Por eso, cuando Apps Script tarda en "despertar" (cold start), los reportes pueden demorar 1-3 segundos.
- **Logística Virgilio NO lee el Sheet directamente del galpón**: va a Internet, llega a Google y baja los datos. Si se corta Internet en la PC, Logística Virgilio tampoco anda.

Cervantes



Tu descripción es correcta. Diferencias técnicas importantes vs Virgilio:

Aspecto	Virgilio (Sheets + Apps Script)	Cervantes (Supabase)
Backend	Google Apps Script + Sheets	Supabase (Postgres real)
Velocidad de escritura	Lenta (cold start 1-3 s)	Rápida (<300 ms)
Concurrencia	Mala — escrituras simultáneas chocan en el Sheet	Buena — Postgres maneja miles/seg
Confirmación de éxito	Mala — Apps Script muchas veces no confirma bien	Buena — la API responde 200/201 o error claro
Cuotas / Límites	Apps Script tiene cuotas diarias y por minuto	Más laxos

Estabilidad histórica	Variable	Más estable
--------------------------	----------	-------------

Conclusión técnica: Cervantes está mejor armado. La arquitectura de Virgilio (Sheets + Apps Script) es la causa principal de que se pierdan mensajes, más allá del WiFi. **Si Virgilio se migrara a Supabase como Cervantes, casi todos los problemas que tenés desaparecerían.**

3) Mensajes que no llegan — Diagnóstico y solución propuesta

El problema en el código actual

Revisando el **index.html original**, dentro de la función **send()** aparece esta porción de código que es la causa raíz del problema:

```
fetch(GOOGLE_SCRIPT_URL, {
  method: "POST",
  body: JSON.stringify({
    legajo: legajoInput.value,
    opcion: selected,
    descripcion: desc[selected],
    texto: texto
  })
}).catch(()=>{}); // <-- silencia CUALQUIER error de red

alert("Reporte enviado correctamente"); // <-- se muestra SIEMPRE,
// haya llegado o no
```

Qué significa esto en castellano: "intentá mandar el reporte; si hay un error de red, ignoralo silenciosamente y no se lo digas a nadie". Y a continuación, **siempre** se le muestra al operario "Reporte enviado correctamente", incluso si la red estaba caída y el mensaje nunca llegó al Google Sheet.

Por qué se pierden mensajes (con este código)

- Si la **señal WiFi se cae** justo en el momento del envío → el fetch falla, el .catch lo silencia, el operario ve "Reporte enviado correctamente" y el reporte **no existe en ningún lado**.
- Si **Apps Script demora más de lo normal** (cold start, cuotas, mantenimiento de Google) y el operario cierra la app antes de que termine → el reporte se pierde.
- Si el operario **cambia de pantalla rápido** después de tocar "Enviar" → el navegador puede cancelar la solicitud antes de enviarla.
- No hay **ningún reintento**: un solo intento fallido = mensaje perdido.
- No hay **ninguna forma de saber**, ni desde el celular ni desde el lado del Sheet, cuántos mensajes se perdieron.

Solución propuesta (a implementar)

La idea es exactamente la que planteás: **guardar cada mensaje localmente antes de mandarlo**, revisar si llegó, y si no, reenviarlo automáticamente cuando vuelva la señal. En la jerga técnica se llama "cola offline con reintentos" (offline-first queue).

La propuesta consiste en agregar al index.html lo siguiente:

- 1 Cola local en localStorage del celular.** Cada reporte se guarda en una lista en el celular **antes** de intentar mandarlo. Si se cierra el navegador, si se corta el WiFi, si se queda sin batería — el reporte sigue ahí.
- 2 Envío con timeout duro de 12 segundos.** Si Apps Script no responde en ese tiempo, el reporte queda marcado como "no enviado" y se reintentará luego.

3 Mensaje real al operario según lo que pasó:

- Si llegó → "Reporte enviado correctamente".
- Si NO llegó → "⚠ Sin conexión. Tu reporte quedó guardado y se reenviará automáticamente cuando vuelva la red".

4 Reintentos automáticos sin que el operario tenga que hacer nada, cuando: (a) vuelve la conexión a Internet, (b) el operario reabre la pestaña, (c) la pestaña vuelve a estar visible, y (d) cada 30 segundos como red de seguridad.

5 sendBeacon al cerrar la pestaña. Es un mecanismo del navegador que entrega los paquetes pendientes en segundo plano aunque la página ya no esté visible. Último recurso para no perder reportes si el operario cierra el navegador.

6 Banner rojo de pendientes en la pantalla de legajo. Si hay reportes que no se pudieron enviar, se ve "⚠ N reporte(s) sin enviar — Tocá para reintentar". Sirve también para forzar el reintento manualmente.

7 ID único por reporte. Cada payload llevaría un id único (timestamp + aleatorio). Esto permite que, opcionalmente, el Apps Script descarte duplicados (si por mala suerte el celular reenvía algo que en realidad ya había llegado).

Con esta solución, el resultado al **final del día** es el mismo que tendrías hoy si todo funcionara perfecto, pero **durante el día** ningún reporte se pierde por problemas de red. Y los que momentáneamente no pudieron salir quedan visibles en la pantalla de legajo para que el operario los pueda forzar.

Cambio mínimo recomendado del lado del Apps Script

Para eliminar el riesgo de duplicados en caso muy raro (el servidor recibe pero el celular no se entera), conviene que el Apps Script descarte por **id** repetido. Un fragmento aproximado dentro de doPost:

```
function doPost(e) {
  var data = JSON.parse(e.postData.contents);
  var id = data.id;
  var cache = CacheService.getScriptCache();
  if (cache.get("rep_" + id)) return; // ya procesado: descartar
  cache.put("rep_" + id, "1", 3600);
  // ... resto del append a la hoja
}
```

Recomendación final (en orden de prioridad)

- 1 Implementar la cola offline + reintentos en el index.html** (Sección 3 de este documento). Sin tocar nada más, esto solo ya resuelve la mayoría de los reportes que hoy se pierden silenciosamente.
- 2 Mejorar el WiFi del galpón:** 2-3 Access Points cableados con el mismo SSID (no repetidores). Inversión baja, mejora notoria de cobertura y estabilidad.
- 3 Evaluar migración de Virgilio a Supabase,** como ya funciona Cervantes. Esto unificaría las dos plantas en la misma arquitectura, eliminaría los problemas crónicos de Apps Script (cold start, cuotas, confirmaciones poco confiables) y dejaría un sistema más predecible.
- 4 Solo si lo anterior no alcanza** (porque tenés cortes de Internet largos y frecuentes), considerar un servidor local en el galpón con base de datos local que sincronice contra la nube en segundo plano. Es la opción más resistente pero también la más cara y compleja de mantener.

Documento generado a partir del archivo index.html original, sin reflejar modificaciones posteriores.